CSD 302

Design and Analysis of Algorithms

Tutorial Problem

Typesetting problem The input is a sequence of n words of lengths $l_1, l_2 \ldots l_n$ measured in characters. We want to print it nicely on a number of lines that can hold a maximum of M characters each. The criterion for "niceness" is as follows. No word can be split across lines with a blank separating words and each line should be as full as possible. The penalty for a trailing space of s is s^3. If s_i is the space left in line i, we want to minimize $\sum_i s_i^3$.
If the penalty function is $\sum_i s_i$, would a greedy approach work ?

<u>Approach:</u>

Let us look at Greedy:

- Put as many words as possible in the current line
- Then move to the next line and so on.

However, as easy as it looks, it may not work for all cases:

<u>Example:</u>

Let the sequence be "*I would like to study for my security quiz*"

I	1
Would	5
Like	4
To	2
Study	5
For	3
My	2
Security	8
Quiz	4

Greedily, we get:

I would	**Empty Spaces: 3**
like to	**Empty Spaces: 3**
study for	**Empty Spaces: 1**
my	**Empty Spaces: 8**
security	**Empty Spaces: 2**
quiz	**Empty Spaces: 6**

Total Error $= 3^3 + 3^3 + 1^3 + 8^3 + 2^3 + 6^3$

$= 27 + 27 + 1 + 512 + 8 + 216$

$=$ **791** ☹

However, a better split could be:

I would	**Empty Spaces: 3**
like to	**Empty Spaces: 3**
study	**Empty Spaces: 5**
for my	**Empty Spaces: 4**
security	**Empty Spaces: 2**
quiz	**Empty Spaces: 6**

Total Error $= 3^3 + 3^3 + 5^3 + 4^3 + 2^3 + 6^3$

$= 27 + 27 + 125 + 64 + 8 + 216$

$=$ **467** 😊

Hence, we can safely say with this counter example that greedy is not the way to go.

Okay, let's get on with it.

We can use Dynamic Programming to solve this problem

Let us use the same string and create a table of size n^2. This $(I,j)^{th}$ entry of this table indicates the cost of storing all the words from I to j in one line, where I and j are indices.

Here is the length table again for reference:

Index	Word	Length
0	I	1
1	Would	5
2	Like	4
3	To	2
4	Study	5
5	For	3
6	My	2
7	Security	8
8	Quiz	4

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
0	729	27	∞	∞	∞	∞	∞	∞	∞
1	-	125	0	∞	∞	∞	∞	∞	∞
2	-	-	216	27	∞	∞	∞	∞	∞
3	-	-	-	512	8	∞	∞	∞	∞
4	-	-	-	-	125	1	∞	∞	∞
5	-	-	-	-	-	343	64	∞	∞
6	-	-	-	-	-	-	512	∞	∞
7	-	-	-	-	-	-	-	8	∞
8	-	-	-	-	-	-	-	-	216

The combinations of words that cannot fit into one line are denoted by infinity.

Clearly, all that's left to do now is to come up with an appropriate recursion. To that end, let us define some notation:

C[j] = Optimised total cost for arranging words from 1 to j in lines.

S(i,j) = Cost of arranging words I to j in one line

So, the recursion can be defined as follows:

$$C[j] = \begin{cases} 0, & \text{if } j = 0 \\ \min\limits_{1 \le i \le j} \left(C[i-1] + S(i, j)\right), & \text{if } j > 0 \end{cases}$$

Yes, I guess that's it.

Space Complexity: **$O(n^2)$**

Time Complexity: $O(n^2) + O(n^2)$ [Cost of building matrix + Recursion]

= **$O(n^2)$**

Greedy for a linear cost function:

- ➔ I reckon it should work as the first thing greedy achieves is that it minimised the number of lines used. And since the number of characters in the string are fixed, we should be able to get to an optimal cost without any problem.